

Multi-player Trivia Tournament

Computer Science 343 - Project 2 Report

Group members and roles

Members	SU number	Role
Thaakir	26479443	Backend, Frontend
Tashreeq	27163245	Database, Backend,
Raez	27445623	Database, Frontend
Deon	26894319	Frontend and Report
Leesan	26186152	Backend, api and Report

Introduction

Our Multi-player Trivia Tournament application represents a modern, real-time, trivia gaming platform designed which allow multiple players from different locations to join and play a trivia match. This web-based application addresses the growing demand for interactive, multiplayer educational entertainment that brings together both a social and a learning environment.

Project Overview

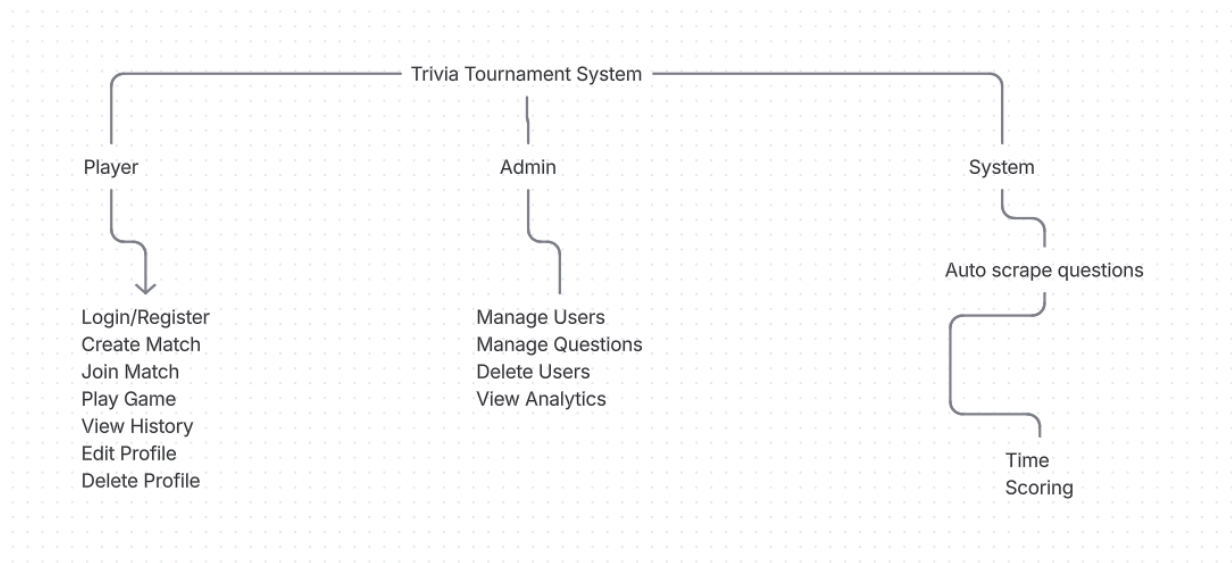
Our application allows authenticated users to create and join trivia matches across multiple different categories. The application supports real-time gameplay with synchronised countdown timers, live score updates, and a comprehensive match history tracking system. The application architecture makes use of a variety of modern web technologies, including React.js with Vite for the frontend, Node.js with Express for the backend, PostgreSQL for data persistence, and WebSocket technology for real-time communication. This technical stack ensures optimal performance, security, and user experience.

Key Motivations

- Educational Value

- Promotes learning through gamification across diverse knowledge domains with a comprehensive set of answers.
- Social Interaction
 - Facilitating competitive learning environments that connect users globally
- Real-time Engagement
 - Delivers synchronous gaming experiences with instant feedback
- Scalability
 - Building a robust platform capable of handling multiple concurrent matches with multiple different uses from different regions.
- Security
 - Implementing comprehensive authentication and authorisation systems

User Case and User Story



Player User Story

Authentication & Profile Management:

- As a player, I want to register with a unique username and email so that I can create a personal account
- As a player, I want my account to be secure by using valid login details, secured by a hashed password
- As a player, I want to be able to utilise a remember me feature to prevent constant login
- As a player, I want to upload an avatar via URL so that I can personalise my profile
- As a player, I want to create a dynamic profiling system which allows for instant editing of my account

- As a player, I want to be able to delete my account and match history so that I can remove my data if needed

Match Management:

- As a player, I want to view a dashboard of all available matches, which will include a section for both current and previous matches
- As a player, I want to be able to search for matches by ID so that I can find a specific game
- As a player, I want to sort matches by most recent activity so that I can find active matches to join
- As a player, I want to be able to filter matches by category so that I can play matches with my preferred category
- As a player, I want to be able to create custom matches so that I can create games with my preferred match settings
- As a player, I want to have the ability to delete matches I created so that I can manage my hosted games

Gameplay:

- As a player, I want to see real-time updates of all players in the match so I can know who is in the lobby
- As a player, I want to receive the questions with a 20-second timer to ensure that gameplay remains fast-paced
- As a player, I want to be able to see the live score updates for a match so I can see where I am in the leaderboard
- As a player, I want to be able to view my match history so that I can track my progress

Admin User Stories

User management

- As an admin, I want to view all registered users so I can keep track of them
- As an admin, I want to have the ability to delete registered users
- As an admin, I want to be able to remove specific users from a specific match

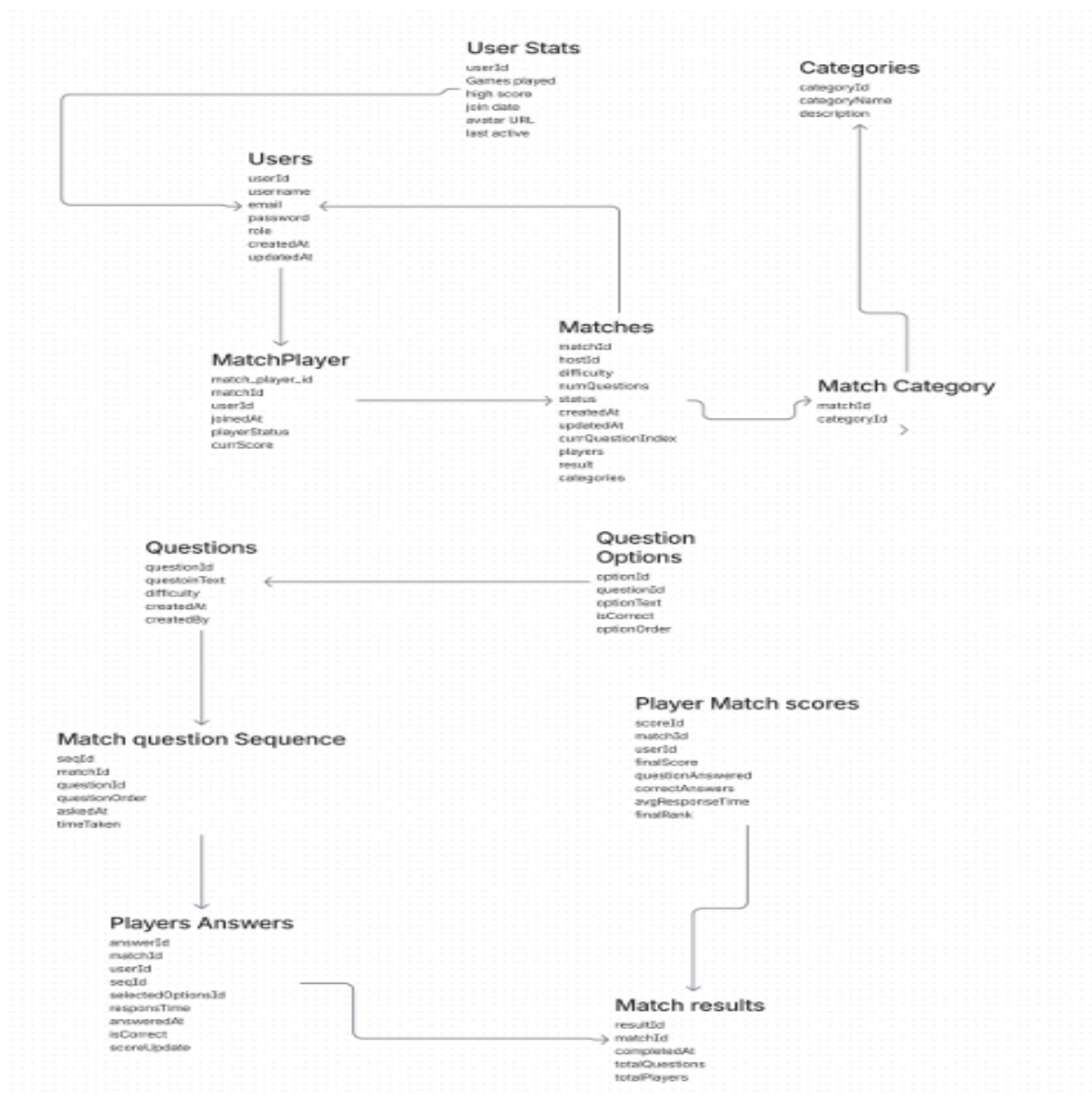
Question Management

- As an admin, I want to be able to view all questions in the database so that I can review the validity of the questions
- As an admin, I want to have the option to search and filter questions to quickly find specific questions

- As an admin, I want to be able to edit and delete existing questions so that I can ensure questions remain correct
- As an admin, I want to be able to manage categories so that I can organise content effectively

Data Modeling

Entity Relationship Diagram



Database schema

Core tables

User table

- Purpose: Stores user authentication data and gaming statistics
- Constraints: Unique username and email addresses
- The password is then hashed, and an avatar image is stored as an external URL

Matchtable

- Purpose: Core match configuration as well as state management
- Key Constraints: Host ID references user. Status follows the defined workflow
- Maximum 7 questions per round, 4 rounds per game

Trivia table

- Purpose: A Question bank which stores questions and answers, along with difficulty level
- Key Constraints: Questions must have an associated answer and difficulty
- Questions obtained via web scraping as well as manual entry

Relationship tables

MatchPlayer Table (Junction)

- Purpose: Resolves the Many-to-Many relationship between Users and Matches
- Key Constraints: Unique constraint which prevents duplicate match joins
- Tracks individual players' performance within a specific match

MatchCategory table (junction)

- Purpose: Resolves the Many-to-Many relationship between Matches and Categories
- Key Constraints: Composite primary key ensures unique match-category pairs
- Matches can span multiple categories for question selection

MultiCat Table (Junction)

- Purpose: Links questions to selected categories through a Many-to-Many relationship
- Key Constraints: Unique category names, which prevents duplication

Game session tables

MatchResults Table

- Purpose: Records both match completion and timing
- Key Constraints: One-to-One relationship with the Match table
- It is only created when the match status becomes COMPLETED

MatchQuestions Table

- Purpose: Tracks the specific questions used in each match
- Key Constraints: Links to both the source question and the match result
- Questions are selected based on match categories and difficulty

MatchAnswer Table

- Purpose: Records player responses with the timing data
- Key Constraints: References user, question, and match for complete context
- 20-second time limit per question, timing affects scoring

FinalScore table

- Purpose: Permanent record of player performance per match
- Key Constraints: Unique constraint ensures one score record per user per match
- Scoring methodology: $100 \text{ points} - (\text{time taken} * 2)$
 - tie-breaking by response time

Normalisation strategy

3NF Compliant Areas:

- The junction tables are used to properly resolve many-to-many relationships
- Makes use of foreign key relationships to maintain referential integrity
- Core entity separation follows the single responsibility principle

Current 3NF Violations:

- User Table: **playerRanking** depends on **gamesPlayed** and **highScore**, which creates a transitivity dependency
- Trivia/MultiCat: Category information could be better normalised, but still functions as intended

Authentication strategy

- Our trivia tournament application implements a JWT-based authentication system with dual-token security, which ensures the user's account is protected with role-based access control.
- Authentication flow
 - User registration validates usernames and email addresses before creating an account.
 - Passwords are then hashed by using a bcrypt with 12 salt rounds

- Then, the authentication generates two JWT tokens upon successful credential validation
 - Access Token → containing user identification → 1 hour expiration
 - Refresh Token → for session persistence → 7-day expiration
- Token scheme implementation
 - The tokenAuthenticate middleware validates JWT tokens on protected routes
 - This ensures that only authenticated users can access sensitive data, such as admin functions
- The system also implements a layered-based authentication system, which validates whether a user is an admin or not.
 - Admin → access to admin functions
 - Player → access to standard game features

API Architecture

Our trivia tournament application implements a comprehensive RESTful API using both Node.js and Express.js, providing structured endpoints for all game functionality. The API follows standard HTTP methods and status codes to ensure predictable client-server communication. A detailed guide for endpoint connection can be found at *"http://localhost:5173/api-docs:"*

- Authentication Endpoints:
 - **POST /api/register** - User registration with password hashing
 - **POST /api/login** - JWT token-based authentication
 - **POST /api/token** - Token validation and refresh
- Match Management Endpoints:
 - **POST /api/createMatch** - Host creates a new trivia match
 - **POST /api/joinMatch** - Players join existing matches
 - **POST /api/getAllMatches** - Retrieve available matches with filtering
 - **POST /api/startMatch** - Host initiates gameplay (triggers question selection)
 - **DELETE /api/deleteMatch/:matchId** - Match deletion with authorization checks
- Question Management Endpoints (Admin-only):
 - **POST /api/createQuestion** - Add questions to the database
 - **GET /api/getAllQuestions** - Administrative question management
 - **POST /api/getQuestion** - Search questions by keyword
 - **DELETE /api/deleteQuestion** - Remove questions with validation

Design pattern

Client-Side Architecture ([React.js](#))

- The frontend implements a modular component structure following React's compositional patterns
- State Management Patterns
 - React Context Pattern
 - Global state management for user authentication and WebSocket connections
 - Custom Hooks Pattern
 - Reusable stateful logic through useLocalStorage for persistent data, such as login
 - Component State Pattern
 - Local UI state management using React's useState and useEffect hooks
- Socket Integration Pattern
 - The SocketContext provider wraps the entire application, enabling real-time communication across all components without prop drilling.

Server-Side Architecture ([Node.js/Express](#))

- Layered Architecture Pattern
 - The backend follows a clear separation of concerns
- Middleware Pattern
 - Authentication Middleware: JWT token validation using tokenAuthenticate
 - CORS Middleware: Cross-origin request handling for frontend integration
 - JSON Parsing Middleware: Request body processing for API endpoint
- Service Layer Pattern
 - Business logic encapsulated in dedicated modules (matchFunctions, userEditor), separating data access from route handling.

Database Integration

- ORM Pattern
 - Prisma provides type-safe database access with automatic query generation and migration management.
- Repository Pattern
 - Database operations are abstracted through Prisma's model-based queries, ensuring consistent data access patterns across the application.

Socket Connections

- We implemented a bidirectional, real-time communication system using [Socket.IO](#) to enable synchronised multiplayer gameplay. The WebSocket architecture ensures that all players in a match experience consistent game state, synchronised timers, and immediate feedback on player actions.
- Server-Side
 - Socket.IO server integrated with Express HTTP server
 - Room-based communication using `match_{matchId}` namespaces
 - Persistent connection tracking via `socketToUser` and `userToSocket` maps
 - Match state management through the `MatchState` interface
- Client-side
 - React Context API (SocketContext) provides global socket access
 - Automatic connection management with reconnection logic
 - Event listeners registered at component mount
 - Connection state tracking `isConnected`
 -

Deployment & Environment, Major Dependencies

Configuration

- Runtime Environment
 - [Node.js](#) version 24+ for backend execution
 - Npm version 11+ for package management and script execution
 - PostgreSQL version 13+ for production-grade data persistence
- Build Tooling
 - Vite version 7+ for frontend development with hot module replacement
 - TypeScript, static type checking across both frontend and backend
 - Prisma database schema management and type-safe ORM

Major Dependencies

- Backend dependencies
 - `@prisma/client`
 - Version: 6.17.0
 - Database ORM and query builder
 - Express
 - Version: 5.1.0
 - Web application framework
 - [socket.io](#)
 - Version: 4.8.1

- Real-time WebSocket communication
- Jsonwebtoken
 - Version: 9.0.2
 - JWT authentication tokens
- Bcrypt
 - Version: 5.1.0
 - Password hashing and security
- Cors
 - Version: 2.8.5
 - Cross-origin resource sharing
- Frontend Dependencies
 - React
 - Version: 18.0.0
 - UI component library
 - React-router-dom
 - Version: 6.0.0
 - Client-side routing
 - socket.io-client
 - Version: 4.8.0
 - WebSocket client integration

Extra Features Implemented, Known Issues, and Contribution Log

Known issues

- The admin page does not display live matches correctly
- User page displays all matches, not just ones in-progress
- Match invite cards: Join lobby and reject invite button does not have functionality
- The send invite is not implemented.

Contribution Log

- AI was used to help explain certain concepts.
- Explained what Swagger UI is and how to do it

References:

Phoenix (no date) *Matrix wallpaper gif*, *Pinterest*. Available at: <https://za.pinterest.com/pin/1196337402506232> (Accessed: 2025).

mar1koff (2022) *Arcade game screen*, *iStock*. Available at:
<https://www.istockphoto.com/vector/arcade-game-screen-copy-space-on-interface-screen-retro-arcade-game-machine-video-gm1432863530-474974712> (Accessed: 17 October 2025).

Beck, H. (no date) *Pac man background*, *Pinterest*. Available at:
<https://za.pinterest.com/pin/16888567434510433> (Accessed: 17 October 2025).

'SFIII: New Generation - Stages' (no date) *The Fighters Generation*. Available at:
<https://www.fightersgeneration.com/games/sf3-p2.html> (Accessed: 17 October 2025).